Simulating Galaxy Encounters with a 2D N-body Code

William Wall

Important note: GitHub repository available at

github.com/WilliamGLWall/N_Body_Simulation_Project

May 26, 2026

Abstract

This project investigates the interaction of two idealised galaxies using a two-dimensional gravitational N-body code. I implemented a Barnes–Hut quadtree algorithm, initialized galaxies from a Plummer-like density profile, evolved the system with fixed and adaptive timesteps, and compared fourth-order Runge–Kutta (RK4) integration with forward Euler integration. I also added differential rotational velocities, energy diagnostics, optimized force evaluation for small particle numbers, and a parameter exploration workflow. The simulations show that the qualitative outcome of a galaxy encounter is controlled mainly by the balance between radial infall, tangential velocity, internal rotation, and impact parameter. The full set of generated figures and videos is supplied in the GitHub repository under `results/`.

1 Introduction

The gravitational N-body problem describes the motion of many particles interacting through Newtonian gravity. In an astrophysical context, the particles may be interpreted as stars, star clusters, or mass elements within a galaxy. For particle i, the acceleration due to all other particles is

$$\ddot{\mathbf{r}}_i = G \sum_{j \neq i} m_j \frac{\mathbf{r}_j - \mathbf{r}_i}{|\mathbf{r}_j - \mathbf{r}_i|^3}. \tag{1}$$

A direct evaluation of Eq. (1) requires all pairwise interactions and therefore scales as $\mathcal{O}(N^2)$. This quickly becomes expensive as the number of particles increases. The Barnes–Hut algorithm reduces this cost by approximating distant groups of particles as a single body located at their centre of mass.

The aim of this project was to build a code capable of simulating two interacting stellar systems and to study how numerical and physical choices influence the resulting merger. The report follows the order of the implementation: first the physical model and initial conditions are described, then the force calculation and integrators, followed by validation, optimization, and parameter experiments. The main code is contained in `N_Body_Opt.ipynb`; all exported outputs are stored in `results/`.

2 Physical Model and Initial Conditions

The simulations were performed in N-body units with $G = 1$. Each galaxy was represented by equal-mass particles. To avoid divergent accelerations during close encounters, I used gravitational softening:

$$\ddot{\mathbf{r}}_i = G \sum_{j \neq i} m_j \frac{\mathbf{r}_j - \mathbf{r}_i}{(|\mathbf{r}_j - \mathbf{r}_i|^2 + \epsilon^2)^{3/2}}, \tag{2}$$

where ϵ is the softening length. Softening is important in this project because a simulation particle represents a finite mass element rather than a literal point star. Without it, very close particle pairs

can dominate the dynamics through unrealistically large accelerations.

The initial positions were sampled from a Plummer-like model,

$$\rho(r) = \frac{3M}{4\pi a^3}\left(1 + \frac{r^2}{a^2}\right)^{-5/2}. \tag{3}$$

This produces a centrally concentrated object with an extended outer halo. The inverse sampling formula used in the code was

$$r = \frac{a}{\sqrt{p^{-2/3} - 1}}, \tag{4}$$

where p is a random number drawn uniformly between 0 and 1. After sampling the radius, a random angular direction was assigned to obtain two-dimensional Cartesian positions.

Two galaxies were initialized independently and then shifted apart. In the final differential-rotation setup, their centres were placed at

$$\mathbf{R}_1 = \left(-\frac{d}{2}, -\frac{b}{2}\right), \qquad \mathbf{R}_2 = \left(\frac{d}{2}, \frac{b}{2}\right), \tag{5}$$

where d is the horizontal separation and b is the impact parameter. The actual initial separation is therefore

$$R_{\text{sep}} = \sqrt{d^2 + b^2}. \tag{6}$$

This distinction became important during parameter exploration: changing d alone does not control the full separation if the impact parameter is large.

The first simulations used only bulk velocities, so the galaxies moved almost directly toward each other. I later implemented differential rotational velocities, using

$$v(r) = \sqrt{\frac{GM(<r)}{r}}, \tag{7}$$

where $M(<r)$ is the enclosed mass. This velocity was applied tangentially around each galaxy centre. A parameter `rotation_scale` was included to reduce or increase the amount of internal rotational support. Large values gave more visible rotation but could also eject loosely bound particles; smaller values produced more compact galaxies.

3 Completion of the Project Tasks

This section gives an overview of how the tasks from the project description were completed in my notebook. The implementation follows the same order as the code: first the force calculation and initial conditions were built, then the time evolution, checks, visualization, energy diagnostics, and parameter experiments were added. All results are saved and organised in the `results/` file as well as in the uploaded notebook.

3.1 Barnes–Hut algorithm in 2D

The Barnes–Hut algorithm was implemented using a two-dimensional quadtree. Each node stores a square bounding box, its total mass, its centre of mass, and up to four child nodes. When a particle is inserted, the tree recursively subdivides until particles are separated into smaller quadrants. I also added safeguards to prevent infinite subdivision when particles are extremely close together or have nearly identical positions.

The force calculation follows the Barnes–Hut opening criterion

$$\frac{L}{r} < \theta, \tag{8}$$

where L is the node width, r is the distance from the target particle to the node centre of mass, and θ is the opening angle. If this condition is satisfied, the whole node is treated as one approximate source. Otherwise, the code opens the node and sums over its children. This is the main approximation that reduces the cost compared with direct summation.

3.2 Initializing two galaxies

I wrote initialization functions for both the basic and differential-rotation galaxy setups. Each galaxy is first sampled from the Plummer-like distribution described in Section 2. The two systems are then shifted so that their centres are separated by a horizontal distance d and, in later experiments, an impact parameter b. The impact parameter was important because it allowed the encounter to move from a head-on collision toward a grazing interaction or fly-by.

The first velocity model used equal and opposite bulk velocities, causing the two galaxies to collide almost directly. I later added differential rotational velocities using Eq. (7). This made the internal structure of each galaxy more dynamic and produced more interesting mergers. The parameter `rotation_scale` was used to tune the amount of internal rotation.

3.3 Solving the N-body problem

The main solver uses RK4 integration. At each timestep, the code evaluates the derivatives four times and combines them to update particle positions and velocities. RK4 was chosen because it is significantly more accurate than Euler for the same timestep, although it is more expensive because each step requires four force evaluations.

I also implemented forward Euler as a comparison method:

$$\mathbf{r}_{n+1} = \mathbf{r}_n + \Delta t\, \mathbf{v}_n, \tag{9}$$

$$\mathbf{v}_{n+1} = \mathbf{v}_n + \Delta t\, \mathbf{a}_n. \tag{10}$$

This provided a useful demonstration of how strongly the integration method affects the physical reliability of the simulation. Both fixed and adaptive timestep routines were implemented. The fixed timestep was used for most controlled comparisons, while the adaptive timestep reduced Δt when the maximum acceleration became large. Adaptive timestepping implementation did help decrease runtime as expected but it lead to less visually appealing simulations and I chose to use fixed time-steps for the majority of my results as well as if in future wanting to use a symplectic integrator it would be compatible.

3.4 Sanity checks

Several sanity checks were included to verify that the code was behaving sensibly. These checks confirmed that positions and velocities were finite, masses were positive, the total particle mass was as expected, the quadtree root mass matched the total particle mass, and computed accelerations did not contain NaN or infinite values. These checks were especially useful when testing the quadtree insertion method and avoiding numerical failures during close encounters.

3.5 Visualization

The notebook includes plotting functions for initial conditions, single frames, multi-frame snapshots, and animations. In the final version, particles from the two galaxies are coloured red and blue on a dark background. This made it much easier to track how material from each galaxy mixed during the merger. The exported results are saved into the repository under `results/figures/` and `results/animations/`. Important examples include `fixed_timestep_rk4_snapshots.png`, `differential_rotation.mp4`, `interesting_showcase.mp4`, and the `playground_case_*.mp4` files.

3.6 Parameter experiments

I varied particle number, timestep, integration method, separation, impact parameter, tangential velocity, radial velocity, and internal rotation strength. I also optimized the force calculation. Although Barnes–Hut is asymptotically faster than direct summation, for the particle numbers used in many runs the vectorized NumPy direct calculation was faster than my pure-Python tree traversal. I therefore added an automatic method selector, using vectorized direct summation below a particle threshold and Barnes–Hut above it.

The physical experiments showed that the outcome depends mainly on angular momentum. Small impact parameter and low tangential velocity produced direct collisions. Moderate impact parameter and tangential velocity produced spiralling mergers. Large impact parameter or large tangential velocity produced disruptive fly-bys, where the galaxies interact but remain at least partly separated.

3.7 Energy conservation

The total energy was calculated as

$$E = K + U, \tag{11}$$

with

$$K = \frac{1}{2}\sum_i m_i |\mathbf{v}_i|^2 \tag{12}$$

and softened potential energy

$$U = -G\sum_{i<j} \frac{m_i m_j}{\sqrt{|\mathbf{r}_i - \mathbf{r}_j|^2 + \epsilon^2}}. \tag{13}$$

This diagnostic directly tests whether the numerical solution behaves like an isolated gravitational system. RK4 approximately conserved total energy over the runs considered, with small drift over long integrations. Euler produced much larger drift, confirming that the integration scheme has a strong effect on physical reliability.

3.8 Interesting and additional simulations

The final notebook includes several presentation-ready cases. The `interesting_showcase` run explores a more asymmetric differential-rotation encounter. The playground section then allows multiple parameter combinations to be run without changing the earlier code. These include a low-angular-momentum collision, a grazing merger, and a high-velocity fly-through. Additionally to investigating and presenting different interesting cases I also looked at a different integrator (Euler Method) and compared it to my RK4 method.

4 Results and Discussion

The exported results are not presented directly in this report because there are many outputs and several are best interpreted as animations. Instead, they are included in the GitHub repository under `results/`. In the following subsections, the most relevant files are listed beside the result being discussed.

4.1 Baseline RK4 simulations

Useful files:

- `results/figures/fixed_timestep_rk4_snapshots.png`
- `results/animations/fixed_timestep_rk4.mp4`

The first successful simulations used two Plummer-like systems with equal and opposite bulk velocities. These runs behaved like nearly head-on collisions. This was expected because the initial velocity was dominated by radial motion, and the two galaxy centres had little angular momentum about their common centre of mass.

In these baseline runs, the galaxies moved almost directly into one another and mixed rapidly. The result was useful for validating the basic solver because it clearly showed mutual gravitational attraction, acceleration during approach, and particle mixing after close passage. However, the morphology was not very galaxy-like: without internal rotation or significant tangential motion, the encounter lacked the curved orbital structure expected in many real mergers.

4.2 Differential rotation and encounter geometry

Useful files:

- `results/animations/differential_rotation.mp4`
- `results/figures/differential_rotation_snapshots.png`

The differential-rotation simulations produced more interesting behaviour. Giving particles tangential velocities around their own galaxy centres made the initial systems less static and gave the encounter a clearer sense of internal structure. The red and blue colouring in the exported animations is particularly useful here because it shows how material from the two galaxies remains partially coherent at first and then gradually mixes during the interaction.

The most important physical parameters were the impact parameter, radial velocity, and tangential velocity. The impact parameter controls how offset the encounter is, while the tangential velocity controls how strongly the galaxies move around one another rather than directly toward one another. When these were moderate, the systems curved around each other before merging and produced asymmetric structures. This is visible in the differential-rotation and interesting-showcase animations.

When the impact parameter or tangential velocity was too small, the result became closer to a direct collision. When either was too large, the systems tended toward a fly-by: the galaxies disturbed and stretched each other, but did not fully merge within the simulated time. This behaviour is consistent with the interpretation that the outcome is mainly controlled by the initial orbital angular momentum.

4.3 Parameter exploration

Useful files:

- `results/animations/interesting_showcase.mp4`
- `results/animations/playground_best_merger.mp4`

The playground simulations were useful for separating different physical regimes. A low-angular-momentum case produced a compact, direct collision. A grazing-merger case produced a more curved interaction and stronger asymmetry. A high-velocity fly-through case showed that sufficiently large tangential velocity can prevent a complete merger, even though the galaxies still gravitationally disturb each other. This is also where I developed my favourite looking and best merger, the exact parameter values I used for this are in the table at the end of this report.

The internal rotation strength also required tuning. If `rotation_scale` was too high, particles in the outer parts of the Plummer distribution could leave their parent galaxy early. Reducing `rotation_scale` kept the galaxies more compact, while increasing the softening length reduced

violent close-particle scattering. The final useful simulations therefore came from balancing internal rotation, softening, radial infall, tangential motion, and impact parameter rather than changing a single parameter alone.

4.4 Energy conservation and integrator choice

Useful files:

- `results/figures/rk4_energy_conservation.png`
- `results/figures/euler_integrator_energy.png` and `results/animations/euler_integrator_comparison.mp4`

The energy diagnostic showed that RK4 approximately conserved total energy, but not perfectly. The fractional drift became more visible over longer integrations and during stronger interactions. This is physically important because, for an isolated gravitational system, total energy should remain constant. The drift is therefore a measure of numerical error from the timestep, force calculation, and integration method.

The Euler comparison made this clearer. Euler was faster because it only evaluates the acceleration once per timestep, but it produced much larger energy errors and less reliable trajectories. RK4 was therefore a better choice for the main simulations, despite being more expensive. However, RK4 is not symplectic, so long-term energy drift is still expected. A natural extension would be to implement Leapfrog or Velocity–Verlet, which are often better suited to long-term gravitational dynamics.

5 Problems Encountered and Role of AI

I encountered a range of problems and errors when exploring this project myself and utilised Gen-AI to assist me in moving forward, explaining and optimising my solutions. In my original notebook code anywhere AI was directly used or cell explicitly written by AI is highlighted. Below I will briefly outline/discuss some explicit examples of its role in this project.

Several practical issues arose during the development of the project. One of the first was making the quadtree insertion robust. If two particles were extremely close together, the tree could keep subdividing into smaller boxes and either become inefficient or fail. AI suggested and helped implementing adding a maximum depth, a minimum node size, and logic to merge unresolved particles inside a leaf node when further subdivision was no longer useful.

A second issue was runtime. The first long simulations using RK4 and Barnes–Hut were slow because each RK4 step requires four force evaluations, and each force evaluation rebuilds or traverses the tree. For the moderate particle numbers used in many of my runs, AI suggested that a vectorized direct-summation calculation was faster than the pure-Python Barnes–Hut implementation. I therefore with guidance and assistance from AI added an optimized direct method and an automatic method selector. This made long parameter experiments much more practical.

There were also presentation and workflow issues. Some animations became too large when too many frames were saved, so I added export controls and organized outputs into a `results/` directory. I also changed the visual style so that particles from the two galaxies are shown in different colours on a dark background, making the mixing process easier to follow in both images and videos.

AI was used as a coding and writing assistant throughout the project. Its main role was to help interpret errors, suggest debugging strategies, explain numerical methods, and help restructure the notebook and report. For example, AI helped explain the energy drift in the RK4 simulations and helped design the export workflow for figures and videos. The final implementation decisions,

parameter choices, interpretation of results, and report structure were still guided by my own testing and understanding of the simulations.

6 Conclusion

I implemented a two-dimensional N-body simulation of interacting galaxies using Barnes–Hut force approximation, Plummer-like initial conditions, RK4 integration, Euler comparison, fixed and adaptive timesteps, energy diagnostics, and parameter exploration. The results show that merger morphology depends strongly on the initial angular momentum, controlled mainly by impact parameter and tangential velocity. Differential rotation made the systems more galaxy-like, while energy diagnostics showed that RK4 was much more reliable than Euler. Future improvements would include a symplectic Leapfrog or Velocity–Verlet integrator, and more realistic galaxy models with separate disk, bulge, and halo components.

A Parameter Values for `playground_best_merger.mp4`

Table 1 lists the main parameters used to produce `results/animations/playground_best_merger.mp4`. This run was selected as one of the clearest merger examples because it combines internal differential rotation with a relatively large impact parameter and tangential velocity, producing a visibly curved encounter rather than a purely head-on collision.

Parameter	Role in the model	Value used
`N_per_galaxy`	Number of particles initialized in each galaxy. Larger values give smoother structure but increase runtime.	150
`d`	Horizontal separation between the two galaxy centres. The true initial separation also depends on the impact parameter.	10.0
`impact_parameter`	Vertical offset between the galaxies. This controls how grazing the encounter is and increases the orbital angular momentum.	12.0
`radial_velocity`	Bulk velocity component driving the galaxies toward one another. Larger values make the encounter more direct.	0.10
`tangential_velocity`	Bulk velocity component causing the galaxies to move around one another. Larger values produce more curved or fly-by-like encounters.	0.22
`rotation_scale`	Scales the internal differential rotational velocity of particles in each galaxy. Lower values keep the galaxies more compact.	0.30
`spin_1`, `spin_2`	Direction of internal rotation for the two galaxies. Opposite signs produce counter-rotating systems.	1, -1
`theta`	Barnes–Hut opening angle. Smaller values are more accurate but slower.	0.50
`eps`	Gravitational softening length. This prevents singular close encounters and reduces violent scattering.	0.05
`t_end`	Final simulation time. Larger values allow the later merger or fly-by behaviour to be seen.	90.0
`dt`	Fixed timestep used by the RK4 integrator. Smaller values improve accuracy but increase runtime.	0.01
`save_every`	Interval, in timesteps, between saved frames. This controls the number of frames in the exported animation.	15
`direct_threshold`	Particle-number threshold below which the optimized vectorized direct force calculation is used.	600

Table 1: Important parameters used for my favourite merger simulation (looked the coolest in my opinion) simulation.